

✓ Final Evolution Lab — Complete Deployment Checklist

Follow this checklist step-by-step to build, deploy, and launch Final Evolution Lab.

Estimated Total Time: 4-7 hours (mostly waiting for builds)

Monthly AWS Budget: ~\$150-400/month (see BUDGET_OPTIMIZATION.md)

Marketing Site: <https://clinquant-figolla-2ad386.netlify.app>

⚠ IMPORTANT COST WARNINGS

AWS charges are ONGOING. Unlike buying a game once, cloud hosting bills you every hour resources are running.

If you forget to shut down your GPU instances, you WILL be billed ~\$12-18/day.

Always set up billing alerts BEFORE deploying. See [Section 7: Cost Management](#) for details.

Table of Contents

1. [Prerequisites & Setup](#) (~1-2 hours)
 2. [Building the UE5 Project](#) (~2-4 hours)
 3. [Deploying AWS Infrastructure](#) (~15-30 minutes)
 4. [Uploading and Deploying the Game](#) (~20-30 minutes)
 5. [Testing and Monitoring](#) (~30 minutes)
 6. [Troubleshooting](#)
 7. [Cost Management](#)
-

Section 1: Prerequisites & Setup

🕒 **Estimated Time: 1-2 hours**

1.1 Required Accounts

- ☐ **AWS Account** — Sign up at <https://aws.amazon.com>
 - You'll need a credit card. New accounts get 12 months of Free Tier (but GPU instances are NOT included in Free Tier).
- ☐ **Epic Games Account** — Sign up at <https://www.unrealengine.com>
 - Required to download Unreal Engine 5.4+
 - Link your GitHub account to access UE5 source code (for Linux builds): <https://www.unrealengine.com/ue-on-github>
- ☐ **GitHub Account** — <https://github.com> (for accessing UE5 source if building on Linux)

1.2 Set Up AWS Billing Alerts (DO THIS FIRST!)

🔔 **Do not skip this step.** Setting billing alerts protects you from unexpected charges.

- ☐ Log in to [AWS Console](#)
- ☐ Go to **Billing & Cost Management** → **Budgets** → **Create budget**
- ☐ Create a **Monthly cost budget** with a limit of **\$400**
- ☐ Set up **alert thresholds**:
 - 50% (\$200) — informational notification
 - 80% (\$320) — warning notification
 - 100% (\$400) — critical alert
- ☐ Add your email address for notifications
- ☐ (Optional) Set up a **Slack webhook** for real-time alerts

1.3 Install Required Software

On Your Local Machine (Windows/macOS recommended for UE5 Editor)

- ☐ **Unreal Engine 5.4+**
 - **Windows/macOS:** Download via [Epic Games Launcher](#)
 - **Linux:** Build from source — use `scripts/ue5_setup/install_ue5_linux.sh`
 - ⚠ **Requires ~100 GB disk space and a dedicated GPU** (NVIDIA GTX 1060+ or equivalent)
 - ⌚ Install time: 30–90 minutes depending on connection speed

- ☐ **AWS CLI v2** — [Install Guide](#)

macOS (Homebrew)

```
brew install awscli
```

Windows (MSI installer)

Download from: <https://awscli.amazonaws.com/AWSCLIV2.msi>

Linux

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -  
o "awscliv2.zip"
```

```
unzip awscliv2.zip && sudo ./aws/install
```

- ☐ **Terraform (v1.5+)** — [Install Guide](#)

macOS

```
brew install terraform
```

Linux

```
sudo apt-get install -y gnupg software-properties-common  
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --  
dearmor -o /usr/share/keyrings/hashicorp-archive-  
keyring.gpg  
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-  
keyring.gpg] https://apt.releases.hashicorp.com  
$(lsb_release -cs) main" | sudo tee  
/etc/apt/sources.list.d/hashicorp.list  
sudo apt update && sudo apt install terraform
```

- ☐ **Node.js 20+** — <https://nodejs.org> (for signalling server & frontend)

- ☐ **Python 3.10+** — <https://www.python.org> (for AI asset pipeline scripts)

☐ **Git** — <https://git-scm.com>

☐ **SSH client** (comes pre-installed on macOS/Linux; use PuTTY on Windows)

1.4 Configure AWS Credentials

☐ Create an **IAM user** with programmatic access in the AWS Console

- Go to IAM → Users → Create User
- Attach the **AdministratorAccess** policy (or a custom policy scoped to EC2, S3, ALB, CloudWatch, ASG, IAM, VPC)
- Create an **Access Key** (CLI access)

☐ Run `aws configure` and enter your credentials:

```
aws configure
# AWS Access Key ID: <your-access-key>
# AWS Secret Access Key: <your-secret-key>
# Default region name: us-west-2
# Default output format: json
```

☐ Verify access:

```
aws sts get-caller-identity
```

You should see your account ID and user ARN.

1.5 Set Up the SSH Key Pair

☐ Copy the provided PEM file to your `~/.ssh/` directory:

```
cp "FinalEvolutionLab.pem" ~/.ssh/FinalEvolutionLab.pem
chmod 400 ~/.ssh/FinalEvolutionLab.pem
```

☐ Import the key pair into AWS (if not already imported):

```
aws ec2 import-key-pair \
  --key-name "FinalEvolutionLab" \
  --public-key-material fileb://<(ssh-keygen -y -f
    ~/.ssh/FinalEvolutionLab.pem) \
  --region us-west-2
```

1.6 Clone the Repository

☐ Clone the project:

```
git clone <your-repo-url> rork-final-evolution-lab
cd rork-final-evolution-lab
```

☐ Install Python dependencies:

```
pip install -r scripts/ai_asset_pipeline/requirements.txt
```

☐ Copy environment template and fill in API keys:

```
cp scripts/ai_asset_pipeline/.env.example .env
# Edit .env with your API keys (MESHY, LUMA, DEEPMOTION)
```

Section 2: Building the UE5 Project

🕒 **Estimated Time: 2-4 hours** (building/cooking can take a while)

2.1 Open the Project in UE5

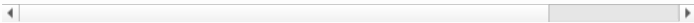
- ☐ Launch Unreal Engine 5.4 Editor
- ☐ Open the project file:

`rork-final-evolution-lab/UnrealStarter/BasketballGame/FinalEvolutionLab.uproject`
- ☐ If prompted about missing modules, click **Yes** to rebuild
- ☐ Wait for shaders to compile (first launch may take 15-30 minutes)

2.2 Import AI-Generated Assets

- ☐ The project includes 49+ AI-generated assets (animations, props, environments) that need to be imported into UE5.
 - ☐ In UE5 Editor, go to **Edit → Editor Preferences → Python**
 - ☐ Enable **Python Editor Script Plugin** if not already enabled
 - ☐ Restart the editor if prompted
 - ☐ Open **Tools → Execute Python Script** (or the Output Log Python console)
 - ☐ Run the master import script:

`UnrealStarter/BasketballGame/EditorPython/fel_import_ai_assets.py`
 - ☐ Then import Elijah Bonds animations:

`UnrealStarter/BasketballGame/EditorPython/fel_import_elijahbonds_`

 - ☐ Verify assets appear in the Content Browser under `/Game/FEL/Generated/`
- Alternative (command-line):** If you have the UE5 environment set up, run:

```
source .ue5_env
./scripts/ue5_setup/import_all_assets.sh
```

2.3 Configure Linux Packaging Settings

- ☐ In UE5 Editor, go to **Edit → Project Settings**
- ☐ Navigate to **Project → Packaging:**
 - Build Configuration: **Shipping** (for production) or **Development** (for testing)
 - ✓ Full Rebuild: **OFF** (unless you changed engine source)
 - ✓ Use Pak File: **ON**
 - ✓ Create compressed cooked packages: **ON**
- ☐ Navigate to **Platforms → Linux:**
 - Target Platform: **Linux**
 - ✓ Cook everything in the project content directory: **ON**
- ☐ Navigate to **Plugins:**
 - ✓ Enable **Pixel Streaming** plugin
 - ✓ Enable **Pixel Streaming Player** plugin

- ☐ Restart the editor after enabling plugins

2.4 Package the Project for Linux Server

☐ Option A: From UE5 Editor

1. Go to **Platforms** → **Linux** → **Package Project**
2. Choose an output directory (e.g., Builds/LinuxServer/)
3. Wait for packaging to complete (1-3 hours)

☐ Option B: From command line (recommended)

```
source .ue5_env
./scripts/ue5_setup/cook_fel_linux_server.sh --config Shipping
```

This will:

- Cook all content for Linux
- Package with Pixel Streaming enabled
- Generate a launch_pixel_streaming.sh script
- Create Docker deployment files

2.5 Verify the Build Output

- ☐ Check that the build output exists:

```
ls -la Builds/LinuxServer/
```

You should see:

- FinalEvolutionLab (binary)
- FinalEvolutionLab/ (content directory)
- launch_pixel_streaming.sh
- Dockerfile and docker-compose.yml

- ☐ Verify the build size (expect 2-8 GB)

- ☐ Test locally if you have a Linux machine with GPU:

```
cd Builds/LinuxServer
./launch_pixel_streaming.sh
```

2.6 Troubleshooting Build Errors

Error	Solution
Missing module: PixelStreaming	Enable Pixel Streaming plugin in Editor → Plugins
Shader compilation failed	Ensure GPU drivers are up to date; try restarting the editor
Out of memory during cook	Close other applications; need at least 16 GB RAM
Linux cross-compilation toolchain not found	Install via UE Editor: Edit → Project Settings → Platforms → Linux → Install SDK
RunUAT not found	Set up .ue5_env file — see .ue5_env.template

Section 3: Deploying AWS Infrastructure

🕒 **Estimated Time: 15-30 minutes**

3.1 Configure Terraform Variables

☐ Navigate to the Terraform directory:

```
cd deployment/aws
```

☐ Copy the example config:

```
cp terraform.tfvars.example terraform.tfvars
```

☐ Edit terraform.tfvars with your settings:

```
nano terraform.tfvars # or use your preferred editor
```

Key settings to update:

Variable	What to Set	Why
key_pair_name	"FinalEvolutionLab"	Must match your imported SSH key
use_spot_instances	true	Saves 60-70% on GPU costs!
enable_scheduled_scaling	true	Turns off instances at night (saves ~50%)
turn_server_enabled	false	Skip TURN server to save ~\$15-120/month
domain_name	Your domain or leave default	For SSL/HTTPS
notification_email	Your email	For billing & health alerts
asg_desired_capacity	1	Start with 1 instance

📖 See [BUDGET_OPTIMIZATION.md](#) for detailed cost scenarios.

3.2 Initialize and Deploy

☐ Initialize Terraform (downloads providers):

```
terraform init
```

✓ You should see: Terraform has been successfully initialized!

☐ Preview what will be created:

```
terraform plan -out=tfplan
```

Review the output carefully. You should see resources like:

- VPC and subnets
- Security groups
- Application Load Balancer
- Auto Scaling Group with GPU launch template
- S3 bucket for builds
- CloudWatch alarms and dashboard
- SNS topic for alerts

☐ Apply the infrastructure:

```
terraform apply tfplan
```

Type yes when prompted. This takes 3-10 minutes.

3.3 Note Down Important Outputs

☐ After terraform apply completes, save these outputs:

```
terraform output
```

Critical outputs to save:

Output	Description	Example
alb_dns_name	Load balancer URL	fel-xxxx.us-west-2.elb.a
streaming_url	Full game URL	https://stream.finalevol
s3_builds_bucket	Where to upload builds	fel-ue5-builds
asg_name	Auto Scaling Group name	final-evolution-lab-gpu-
cloudwatch_dashboard_url	Monitoring dashboard	AWS Console URL
ssh_command	How to SSH into instances	ssh -i ... ubuntu@<IP>

☐ Save outputs to a file for reference:

```
terraform output -json > ../../terraform_outputs.json
```

3.4 Verify Infrastructure

☐ Check the ALB is responding:

```
curl -I http://$(terraform output -raw alb_dns_name)
```

☐ Verify the Auto Scaling Group:

```
aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$(terraform output -raw asg_name)" \
  --query "AutoScalingGroups[0].\
    {Min:MinSize,Max:MaxSize,Desired:DesiredCapacity,Instances:Instances[*].Ins\
    }" \
  --region us-west-2
```

☐ Check the S3 bucket exists:

```
aws s3 ls s3://$(terraform output -raw s3_builds_bucket) --region
us-west-2
```

3.5 Set Up Additional Billing Alarms via Terraform

Terraform already creates CloudWatch alarms. Verify they're active:

- ☐ Check SNS subscription confirmation email (check your inbox + spam)
- ☐ Click the confirmation link in the email
- ☐ Verify in CloudWatch Console: **Alarms** → **All alarms** — you should see FEL-related alarms

Section 4: Uploading and Deploying the Game

🕒 **Estimated Time: 20-30 minutes**

4.1 Upload the Build to S3

- ☐ Navigate to the project root:

```
cd /path/to/rork-final-evolution-lab
```

- ☐ Use the build-and-upload script:

```
./deployment/scripts/build-and-upload.sh
```

This script:

- Compresses the Linux server build
- Uploads it to the S3 bucket
- Tags the upload with a version number

Manual upload alternative:

```
BUCKET=$(cd deployment/aws && terraform output -raw
s3_builds_bucket)
aws s3 sync Builds/LinuxServer/ s3://$BUCKET/builds/latest/
--region us-west-2
```

- ☐ Verify the upload:

```
BUCKET=$(cd deployment/aws && terraform output -raw
s3_builds_bucket)
aws s3 ls s3://$BUCKET/builds/ --recursive --human-readable --
region us-west-2
```

4.2 Deploy to EC2 Instances

- ☐ Run the fleet deployment script:

```
./deployment/scripts/deploy-fleet.sh
```

This will:

- Find all running GPU instances in the ASG
- SSH into each instance
- Download the build from S3
- Start the Pixel Streaming server
- Verify the game is running

Manual deployment (single instance):

```
# Get instance IP
INSTANCE_IP=$(aws ec2 describe-instances \
  --filters "Name=tag:Project,Values=final-evolution-lab" \
    "Name=instance-state-name,Values=running" \
  --query "Reservations[0].Instances[0].PublicIpAddress" \
  --output text --region us-west-2)

# SSH and deploy
ssh -i ~/.ssh/FinalEvolutionLab.pem ubuntu@$INSTANCE_IP
# On the instance:
aws s3 sync s3://fel-ue5-builds/builds/latest/ ~/game/ --
  region us-west-2
chmod +x ~/game/launch_pixel_streaming.sh
~/game/launch_pixel_streaming.sh &
```

4.3 Configure SSL/HTTPS (Optional but Recommended)

SSL is required for WebRTC to work in most browsers (they require HTTPS origins).

☐ Option A: Using AWS Certificate Manager (free)

1. Go to AWS Console → Certificate Manager → Request certificate
2. Request a public certificate for your domain (e.g., stream.finalevolutiongroup.com)
3. Validate via DNS (add the CNAME record to your domain)
4. Update terraform.tfvars with the certificate ARN
5. Run terraform apply to attach it to the ALB

☐ Option B: Using the setup script

```
./deployment/scripts/setup-ssl.sh
```

☐ Option C: Using Let's Encrypt on the instance

```
ssh -i ~/.ssh/FinalEvolutionLab.pem ubuntu@$INSTANCE_IP
sudo certbot --nginx -d stream.finalevolutiongroup.com
```

4.4 Verify Game is Running

☐ Check the health endpoint:

```
ALB_URL=$(cd deployment/aws && terraform output -raw
  alb_dns_name)
curl http://$ALB_URL/healthz
```

Expected response: {"status": "healthy"}

☐ Run the health check script:

```
./deployment/scripts/health-check.sh
```

Section 5: Testing and Monitoring

🕒 **Estimated Time: 30 minutes**

5.1 Access the Game

- ☐ Open a browser and navigate to:

`https://stream.finalevolutiongroup.com`

(or the ALB DNS name if SSL is not configured:

`http://<alb_dns_name>`)

- ☐ You should see the Pixel Streaming player interface
- ☐ Click to connect — the UE5 game should render in your browser

5.2 Test Pixel Streaming Functionality

- ☐ **Video quality:** Verify smooth rendering (should be 30–60 FPS)
- ☐ **Input:** Test mouse/keyboard/touch inputs are responsive
- ☐ **Audio:** Verify game audio is playing through the browser
- ☐ **Game modes:** Try switching between game modes:
 - Basketball H2H
 - Soccer
 - Karate H2H
 - Exercise/Training modes
- ☐ **Latency:** Input-to-display latency should be <100ms for good experience

5.3 Run Automated Tests

- ☐ Run the deployment test suite:

```
./scripts/test_deployment.sh
```

This checks:

- Signalling server health
- Game modes API (expecting 16+ modes)
- WebSocket connectivity
- Frontend accessibility
- Docker containers (if using Docker deployment)

- ☐ Run the game modes test:

```
./scripts/test_game_modes.sh
```

Verifies all 17 game modes are registered and accessible.

5.4 Monitor with CloudWatch

- ☐ Open the CloudWatch Dashboard:

```
cd deployment/aws && terraform output -raw  
cloudwatch_dashboard_url
```

(Copy the URL and open in your browser)

- ☐ Key metrics to watch: | Metric | Healthy Range | Alert Threshold |
| |---|:-----|:-----| | GPU Utilization | 20-80% | >90% sustained | | CPU Utilization | 10-60% | >80% sustained | | Memory Used | <80% | >90% | | Network In/Out | Varies | Sudden spikes | | Active Connections | 1-10+ | >50 (scale up) | | ALB 5xx Errors | 0 | >5/minute |

- ☐ Run the GPU monitoring script on an instance:

```
ssh -i ~/.ssh/FinalEvolutionLab.pem ubuntu@$INSTANCE_IP
./deployment/scripts/monitor-gpu.sh
```

5.5 Check Current Costs

- ☐ Go to [AWS Cost Explorer](#)
 - ☐ Filter by tag: Project = final-evolution-lab
 - ☐ Check daily run rate — multiply by 30 for estimated monthly cost
 - ☐ Verify you're within the \$400/month budget
-

Section 6: Troubleshooting

Common Issues and Solutions

● “Cannot connect to Pixel Streaming”

1. Check if the game server is running:

```
ssh -i ~/.ssh/FinalEvolutionLab.pem ubuntu@$INSTANCE_IP
ps aux | grep FinalEvolution
```

2. Check the signalling server:

```
curl http://$INSTANCE_IP:8888/healthz
```

3. Check security groups allow ports 80, 443, 8888, 8889 (TCP) and 19302-19303 (UDP for TURN)

4. Check browser console (F12) for WebRTC errors

● “Spot instance terminated unexpectedly”

- This is normal with spot instances — AWS reclaims them when demand spikes
- The Auto Scaling Group will automatically launch a replacement
- Check ASG events: `aws autoscaling describe-scaling-activities --auto-scaling-group-name <asg-name> --region us-west-2`
- Consider setting `on_demand_base_capacity = 1` in `terraform.tfvars` for a reliable base instance

● “Build upload failed / S3 access denied”

```
# Verify your AWS credentials
```

```
aws sts get-caller-identity
```

```
# Check S3 bucket policy
```

```
aws s3api get-bucket-policy --bucket fel-ue5-builds --region us-west-2
```

```
# Try uploading a test file
```

```
echo "test" | aws s3 cp - s3://fel-ue5-builds/test.txt --region us-west-2
```

● “terraform apply errors”

```
# Check for state issues
```

```
terraform refresh
```

```
# If state is corrupted, import existing resources
terraform import <resource_address> <resource_id>
```

```
# See all resources in state
terraform state list
```

● “UE5 Editor crashes during packaging”

- Ensure at least 32 GB RAM available
- Update GPU drivers to latest version
- Try packaging from command line instead (see Section 2.4 Option B)
- Check logs: Saved/Logs/Cook.log

Where to Find Logs

Log	Location
UE5 Build logs	Builds/LinuxServer/logs/ and Saved/Logs/
Terraform logs	Set TF_LOG=DEBUG before running commands
EC2 instance logs	ssh into instance → /var/log/cloud-init-output.log
Game server logs	On instance → ~/game/FinalEvolutionLab/Saved/Logs/
Signalling server	On instance → journalctl -u signalling or screen/tmux logs
CloudWatch Logs	AWS Console → CloudWatch → Log Groups → /fel/
Deployment scripts	logs/ directory in project root

How to Rollback Deployments

```
# Rollback to previous build version
BUCKET=$(cd deployment/aws && terraform output -raw
s3_builds_bucket)
aws s3 ls s3://$BUCKET/builds/ --region us-west-2
# Find the previous version and deploy it
aws s3 sync s3://$BUCKET/builds/<previous-version>/ ~/game/ --region
us-west-2
```

```
# Rollback Terraform changes
cd deployment/aws
terraform plan # Review current state
terraform apply -target=<specific_resource> # Apply selectively
```

```
# Full infrastructure rollback
git log --oneline deployment/aws/ # Find the commit to revert to
git checkout <commit> -- deployment/aws/
terraform apply
```

● Emergency Shutdown

If costs are spiraling or something is broken, **shut everything down immediately**:

```
# Option 1: Scale ASG to zero (stops all GPU instances)
aws autoscaling update-auto-scaling-group \
--auto-scaling-group-name "$(cd deployment/aws && terraform output
-raw asg_name)" \
```

```

--min-size 0 --max-size 0 --desired-capacity 0 \
--region us-west-2

# Option 2: Destroy ALL infrastructure (nuclear option)
cd deployment/aws
terraform destroy
# ⚠ This deletes EVERYTHING including S3 data. Make sure you have
  backups.

# Option 3: Stop individual instances (preserves data)
INSTANCE_ID=$(aws ec2 describe-instances \
  --filters "Name=tag:Project,Values=final-evolution-lab"
             "Name=instance-state-name,Values=running" \
  --query "Reservations[*].Instances[*].InstanceId" --output text --
            region us-west-2)
aws ec2 stop-instances --instance-ids $INSTANCE_ID --region us-west-
2

```

Section 7: Cost Management

Daily Cost Monitoring Routine

- ☐ **Week 1:** Check [AWS Cost Explorer](#) **daily**
- ☐ **After Week 1:** Check at least **twice per week**
- ☐ Set a calendar reminder to review costs

How to Estimate Your Current Monthly Spend

```

# Quick cost check via CLI
aws ce get-cost-and-usage \
  --time-period Start=$(date -d "first day of this month" +%Y-%m-
    %d),End=$(date +%Y-%m-%d) \
  --granularity MONTHLY \
  --metrics BlendedCost \
  --filter '{"Tags":{"Key":"Project","Values":["final-evolution-
    lab"]}}' \
  --region us-east-1

```

When to Scale Down

Situation	Action
No users playing	Scale ASG to 0 (--desired-capacity 0)
Low traffic periods	Use scheduled scaling (already configured in Terraform)
Demo/testing only	Run instances only when needed, shut down after
Over budget mid-month	Scale to 0 and switch to on-demand for fewer hours

How to Pause/Stop Instances

```

# Pause (scale to zero – no GPU costs while paused)
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name "<asg-name>" \
  --min-size 0 --max-size 0 --desired-capacity 0 \

```

```
--region us-west-2

# Resume (scale back up)
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name "<asg-name>" \
  --min-size 1 --max-size 2 --desired-capacity 1 \
  --region us-west-2
```

⚠ **Note:** Even when instances are stopped, you still pay for: -
Application Load Balancer (~\$22/month) - S3 storage (~\$1-
5/month) - Elastic IPs not attached to running instances
(~\$3.60/month)

To eliminate ALL costs, run terraform destroy.

Budget Alerts Setup Summary

Alert Level	Threshold	Action
🔔 Info	\$200 (50%)	Review spending trend
🔔 Warning	\$320 (80%)	Consider scaling down or turning off at night
🔔 Critical	\$400 (100%)	Immediately scale to 0 or shut down non- essential resources

📖 For the full cost breakdown and optimization strategies,
see **BUDGET_OPTIMIZATION.md**

Quick Reference Card

What	Command / URL
Marketing site	https://clinquant-figolla-2ad386.netlify.app
Game URL	https://stream.finalevolutiongroup.com (after deployment)
SSH to instance	ssh -i ~/.ssh/FinalEvolutionLab.pem ubuntu@<IP>
Scale to 0	aws autoscaling update-auto-scaling-group -- auto-scaling-group-name <asg> --desired- capacity 0 --min-size 0 --max-size 0 --region us-west-2
Scale to 1	aws autoscaling update-auto-scaling-group -- auto-scaling-group-name <asg> --desired- capacity 1 --min-size 1 --max-size 2 --region us-west-2
Check costs	AWS Cost Explorer
CloudWatch	Run cd deployment/aws && terraform output - raw cloudwatch_dashboard_url
Destroy all	cd deployment/aws && terraform destroy
Budget guide	deployment/docs/BUDGET_OPTIMIZATION.md

Full deploy guide [DEPLOYMENT_GUIDE.md](#)

Last updated: April 2, 2026